CS 422 Assignment 2: iPerf, TCP Statistics, and Congestion Control

Joseph Getachew, Kushagr Khanna, Isaac Hallman

October 21, 2026

Repository: https://github.com/krash3125/cs422-assignment-2. Code links point at commit `725adf6`.

Host. Ubuntu 24.04.5 VM (Multipass on an Apple M1 Pro Mac), kernel **`6.8.0-147-generic`**. Docker (`FROM ubuntu:24.04`) with `--network host --cap-add NET_ADMIN`. `tcp_available_congestion_control = reno cubic bbr` (after `sudo modprobe tcp_bbr`), default `cubic`, qdisc `fq_codel`, `tcp_wmem = 4096 16384 4194304`.

Command. `./run.sh -n 10 -t 30 -i 0.2 -r 3 --seed 422`. Raw data, plots and tables: `results/20261021-082724/`.

1. iPerf Throughput Application

(a) Python iperf3 client

`iperf_client.py` uses only `socket`, `select`, `struct` and `json`; the iperf3 binary is never called. Steps in `run_test`:

Step	Implementation
(i) control connection	TCP connect to `host:port`, send a 37-byte random cookie.
(ii) JSON parameter exchange	On `PARAM_EXCHANGE (9)`, send the parameters below as 4-byte length + JSON.
(iii) data connection	On `CREATE_STREAMS (10)`, open a second socket, set `TCP_CONGESTION` before `connect()`, send the cookie, read the algorithm back.
(iv) transmission	After `TEST_START (1)` / `TEST_RUNNING (2)`, `send_loop` sends 128 KiB blocks for the duration.
(v) termination	`TEST_END (4)`; on `EXCHANGE_RESULTS (13)` exchange results JSON; on `DISPLAY_RESULTS (14)` send `IPERF_DONE (16)` and close.

```
{"tcp": true, "omit": 0, "time": 30, "num": 0, "blockcount": 0, "parallel": 1,
 "len": 131072, "pacing_timer": 1000, "client_version": "3.17"}
```

Robustness. Each failure becomes a typed `status`, not an exception, so one bad server never stops the pipeline:

Situation	Status
Non-responsive / overloaded server (no state byte in 15 s; busy)	`CTRL_TIMEOUT`, `ACCESS_DENIED`
Server rejects parameters (`SERVER_ERROR` + `i_errno`)	`PARAM_REJECTED`, `SERVER_ERROR`
Connection timeout / refused / DNS	`CONNECT_TIMEOUT`, `CONNECT_REFUSED`, `DNS_FAIL`
Premature termination (EOF/reset)	`PREMATURE_CLOSE`, `SERVER_TERMINATE`

Transient failures are retried (`run_with_retries`). All 90 tests completed against standard iperf3 servers; server byte counts matched `tcpi_bytes_acked` within 0.07%.

(b) Destination selection

`select_destinations` shuffles `servers.csv` (183 servers) with a seeded RNG, probes each candidate, replaces failures up to `--max-attempts` (40), and records the shortfall in `destinations.json`.

#	Destination	IP	Port	Site	min RTT (ms)
1	speedtest.wdc2.us.leaseweb.net	23.105.9.80	5209	Washington, US	26.2
2	speedtest.dal13.us.leaseweb.net	209.58.159.68	5207	Dallas, US	27.0
3	66.35.22.79	66.35.22.79	30003	Ashburn, US	27.2
4	84.17.41.11	84.17.41.11	5201	Seattle, US	52.7
5	iperf3.moji.fr	45.147.210.189	5232	Paris, FR	96.9
6	iperf-ams-nl.eranium.net	217.18.95.142	5202	Amsterdam, NL	102.2
7	iperf.worldstream.nl	185.182.195.76	5203	Naaldwijk, NL	104.7
8	89.187.165.1	89.187.165.1	5201	Zürich, CH	116.7
9	speedtest.syd12.au.leaseweb.net	23.106.244.70	5202	Sydney, AU	180.7
10	89.187.162.1	89.187.162.1	5201	Singapore, SG	207.7

10 of 10 selected, shortfall 0.

(c) Goodput

Every 200 ms, `send_loop` reads `TCP_INFO` with `getsockopt()` and `make_sample` computes

$$\text{goodput}(t_k) = \frac{\texttt{tcpi_bytes_acked}(t_k) - \texttt{tcpi_bytes_acked}(t_{k-1})}{t_k - t_{k-1}} \cdot 8 \text{ [bit/s]}$$

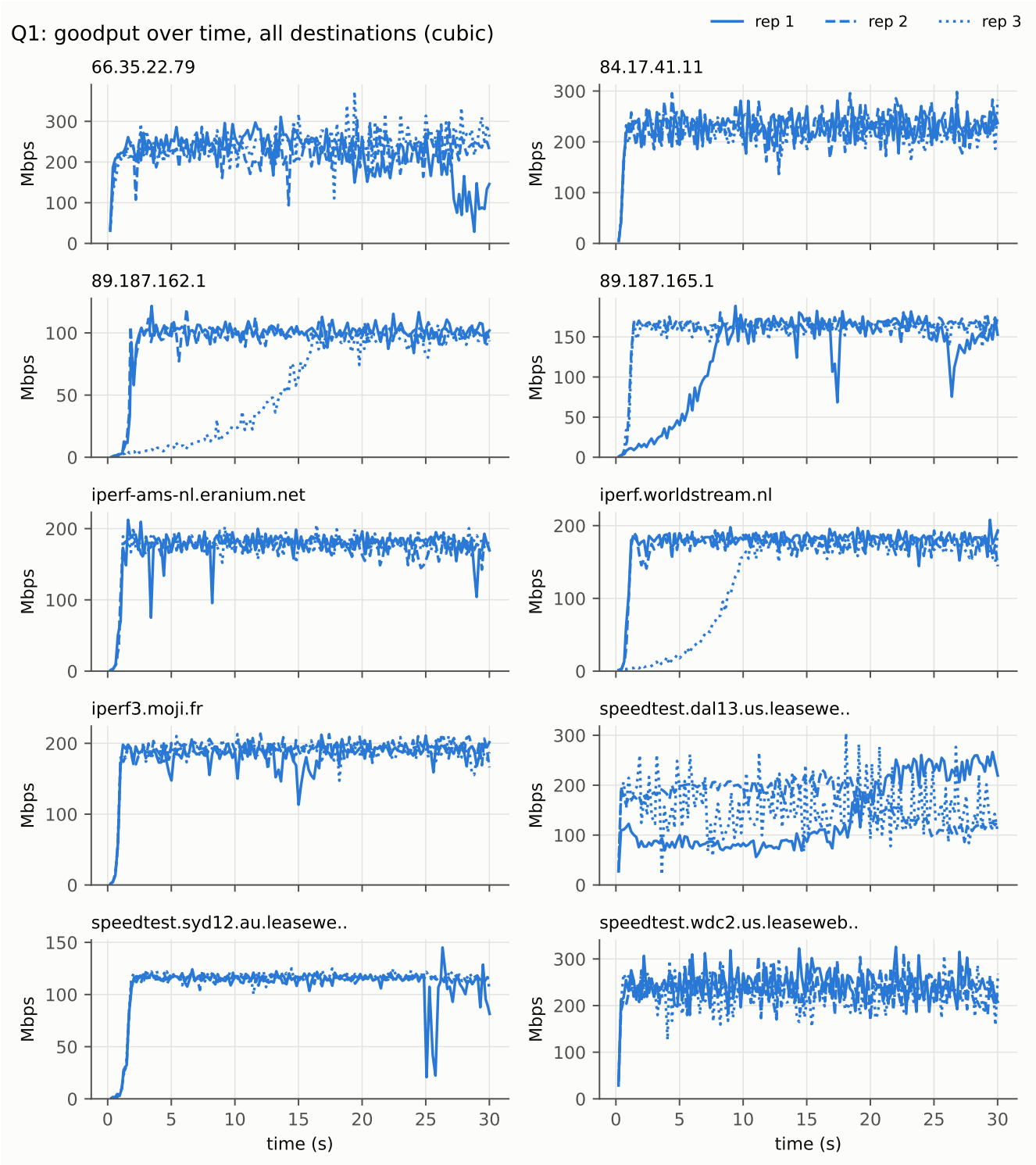


Figure 1: Q1 goodput, CUBIC.

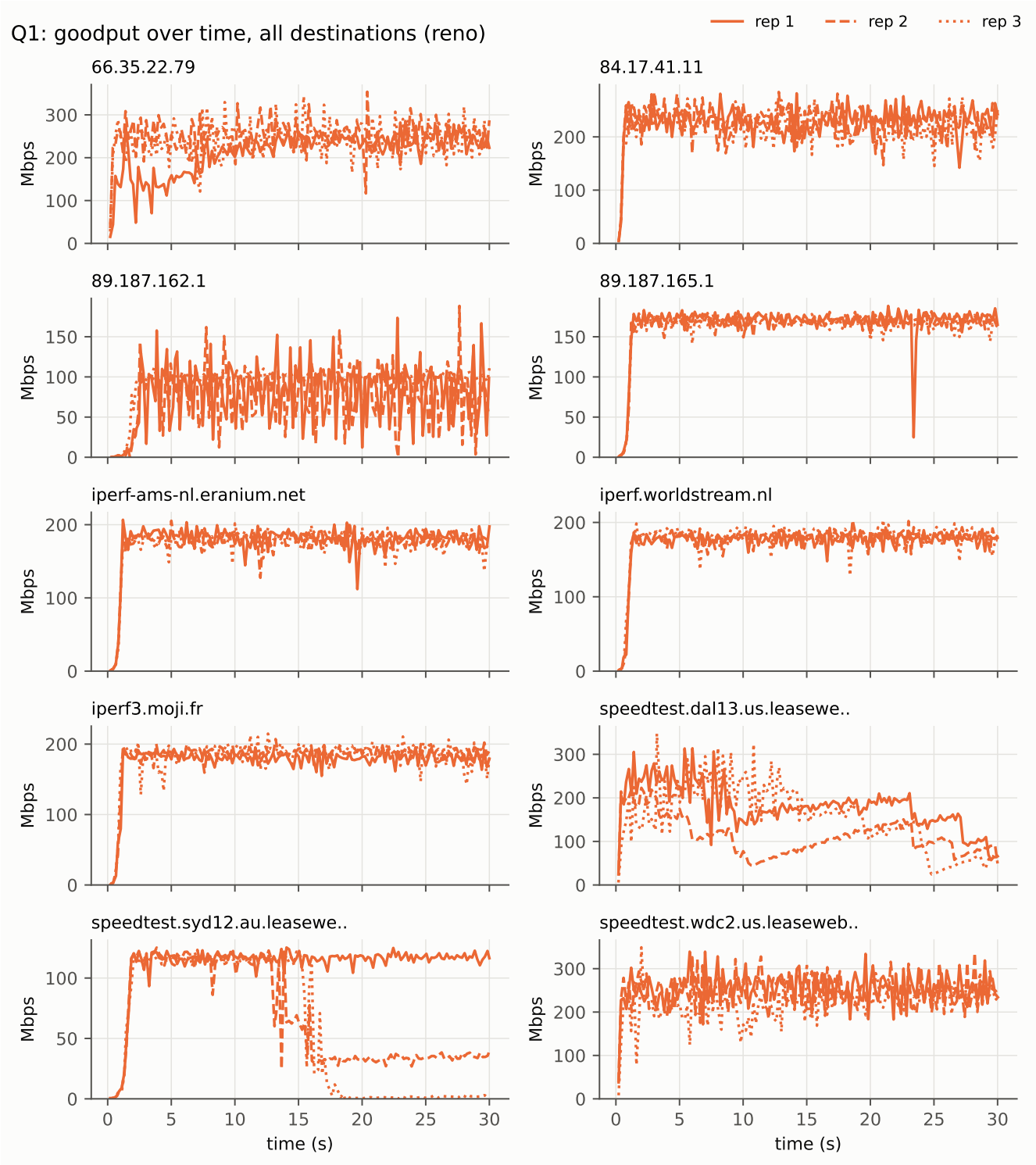


Figure 2: Q1 goodput, Reno.

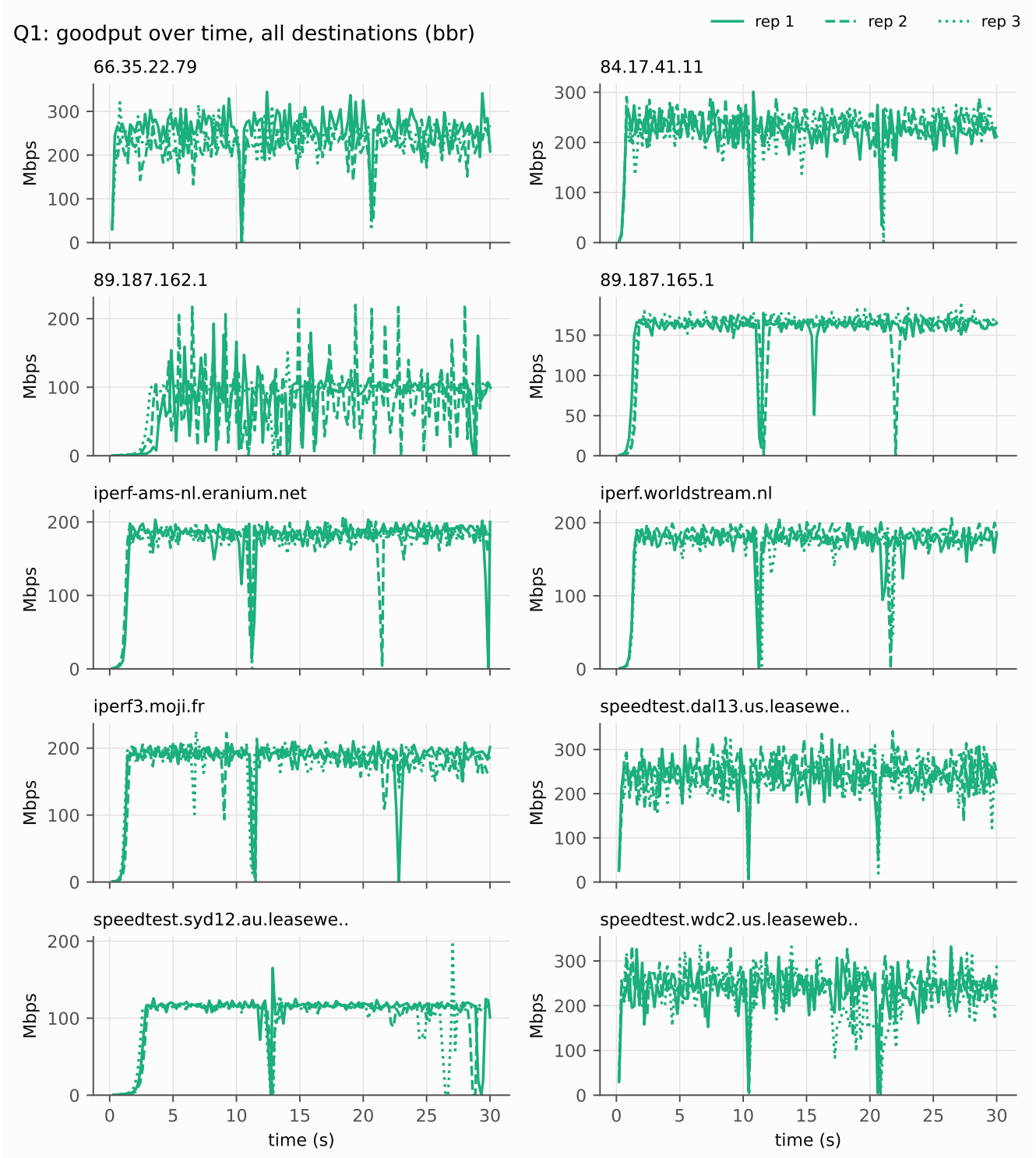


Figure 3: Q1 goodput, BBR.

Goodput over all 200 ms samples of the 3 repetitions (Mbps):

Destination (min RTT)	CC	min	median	mean	p95
Washington (26 ms)	cubic	28.7	233.5	233.7	289.3
	reno	8.1	242.4	241.8	302.2

Destination (min RTT)	CC	min	median	mean	p95
	bbr	0.4	241.2	234.5	306.8
Dallas (27 ms)	cubic	20.3	152.8	154.4	246.8
	reno	5.7	149.2	149.5	260.4
	bbr	1.5	244.5	240.1	304.8
Ashburn (27 ms)	cubic	28.7	234.3	228.4	291.9
	reno	15.4	241.1	235.3	296.5
	bbr	1.2	245.8	239.7	303.1
Seattle (53 ms)	cubic	4.4	229.1	225.0	266.8
	reno	3.8	227.9	224.6	267.1
	bbr	0.5	228.8	223.6	269.9
Paris (97 ms)	cubic	1.1	190.1	184.7	205.5
	reno	0.5	185.9	180.4	198.9
	bbr	0.5	189.9	178.6	205.0
Amsterdam (102 ms)	cubic	1.0	178.7	172.7	193.6
	reno	0.6	181.3	175.1	194.5
	bbr	0.5	184.5	174.0	197.8
Naaldwijk (105 ms)	cubic	0.6	176.6	158.8	190.4
	reno	1.0	180.6	174.8	192.2
	bbr	0.3	179.9	170.2	195.4
Zürich (117 ms)	cubic	0.5	162.7	148.1	173.8
	reno	0.5	169.5	164.1	179.7
	bbr	0.3	165.5	155.8	176.3
Sydney (181 ms)	cubic	0.4	115.8	110.0	121.4
	reno	0.0	113.3	78.9	121.6
	bbr	0.0	115.0	102.4	120.6
Singapore (208 ms)	cubic	0.0	97.9	82.7	107.5
	reno	0.0	97.4	84.3	124.1
	bbr	0.0	97.1	83.1	146.4

2. TCP Stats Tracing During Transfer

CUBIC (kernel default); `cc_used = cubic` in every Q2 sample.

(a) Periodic TCP statistics

Decoding. `tcp_info.py` unpacks `struct tcp_info` (`include/uapi/linux/tcp.h`) with `decode` until the returned 248 bytes run out (a shorter struct on an older kernel is never misread).

Offset	Type	Field	Logged as
1	u8	`tcpi_ca_state`	`ca_state` (3 = Recovery, 4 = Loss)
68 / 72	u32	`tcpi_rtt` / `tcpi_rttvar` (µs)	`srtt_ms`, `rttvar_ms`
80	u32	`tcpi_snd_cwnd` (segments)	`cwnd_pkts`
100	u32	`tcpi_total_retrans`	`retrans_pkts_delta`
120	u64	`tcpi_bytes_acked`	`bytes_acked`, goodput
148	u32	`tcpi_min_rtt` (µs)	`min_rtt_ms`
240	u16	`tcpi_total_rto`	`rto_events_delta`

Interval measurements. Counters (bytes acked, retransmits, timeouts) are differenced between consecutive samples and divided by the measured time gap; cwnd, RTT and rates are instantaneous.

Loss proxy. `retrans_pkts_delta` = segments retransmitted in the interval (from `tcpi_total_retrans`); `ca_state` and timeouts are logged alongside.

Storage. `samples.csv`, one row per sample with `run_id`, `rep`, destination, `cc_used` and the metrics; units are in the column names (`_bps`, `_ms`, `_pkts`) and `metadata.json`.

(b) Visualization

Representative destination (`pick_rep_dest`, median CUBIC goodput): **Amsterdam** (min RTT 102 ms). Time series = median run; scatters = all 3 runs.

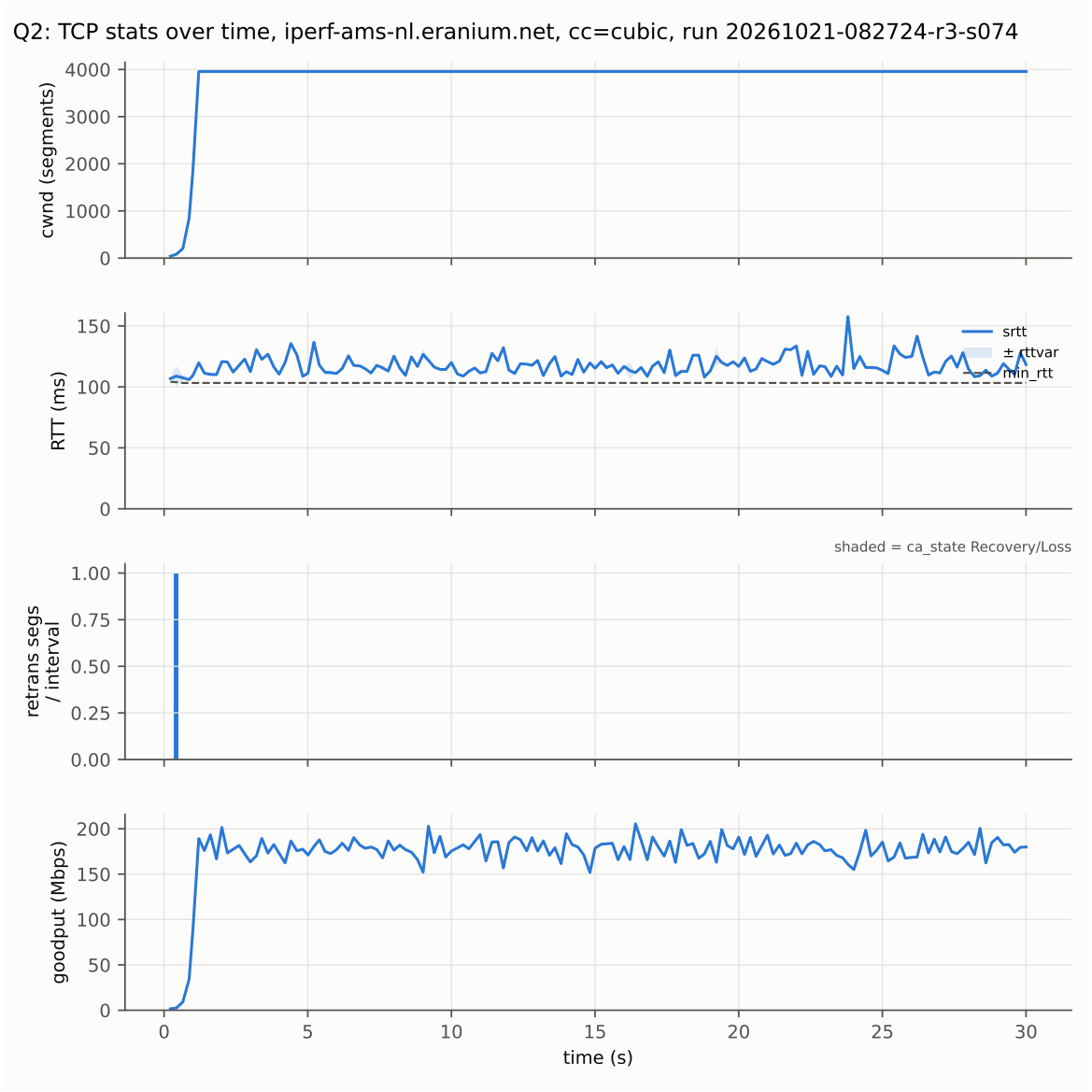


Figure 4: Q2 time series, CUBIC to Amsterdam: cwnd, srtt (dashed = min_rtt), retransmits per 200 ms (shaded = Recovery/Loss), goodput.

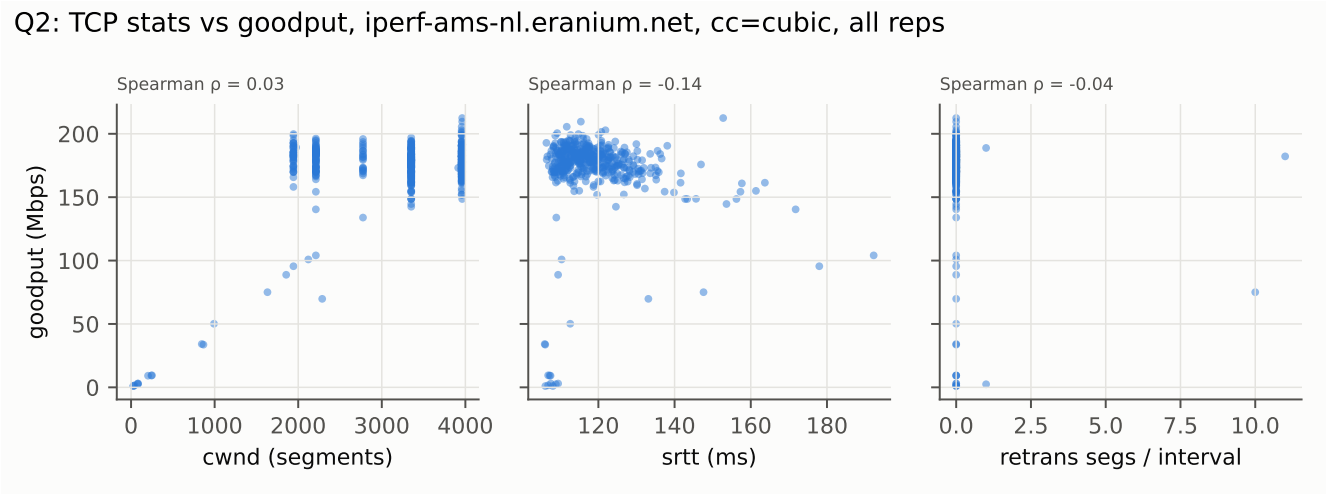


Figure 5: Q2 scatter, all 3 CUBIC runs to Amsterdam.

(c) Observations

- **cwnd**: unacknowledged segments allowed in flight. Below the BDP, raising cwnd raises goodput; above it the link is full and extra window only builds queue.
- **srtt / min_rtt**: srtt − min_rtt is the queueing delay; a rising srtt with flat goodput means a growing queue.
- **Retransmits** (loss proxy): non-zero means a buffer overflowed; CUBIC then cuts cwnd ×0.7, the queue drains, and goodput dips.
- **Goodput**: rate the receiver acknowledges new data.

In the trace, cwnd grows to ~3950 segments in 1.3 s of slow start, then stays flat for 29 s while goodput holds at ~175 Mbps (srtt 116.6 ms vs min_rtt 102.7 ms); cwnd, retransmits and srtt all correlate near zero with goodput ($\rho = 0.03$ / -0.04 / -0.14).

Anomalies.

- *Send-buffer limited, not cwnd-limited.* Median in-flight is 2.53 MB, far below cwnd, and 72% of samples are flagged `app_limited`: the 4 MiB `tcp_wmem` send buffer caps unacked data, so cwnd stays flat. A 32 MiB rerun (`results/wmem32m/`) made cwnd the limit — the CUBIC sawtooth appeared and cwnd–goodput ρ rose to 0.54.
- *Early slow-start exit (HyStart).* Singapore rep 3 left slow start at 59 segments with no loss, reaching only 57.5 Mbps vs 94–96 Mbps in the other reps.

3. Comparing CUBIC, Reno and BBR, and a New Design

(a) Comparison and operating regimes

Method. Same 10 destinations, 30 s, 200 ms sampling for all algorithms; algorithm set before `connect()` and read back (`cc_used = cc_requested` in every test). **3 repetitions**, order rotated each rep (rep 1 cubic→reno→bbr; rep 2 reno→bbr→cubic; rep 3 bbr→cubic→reno).

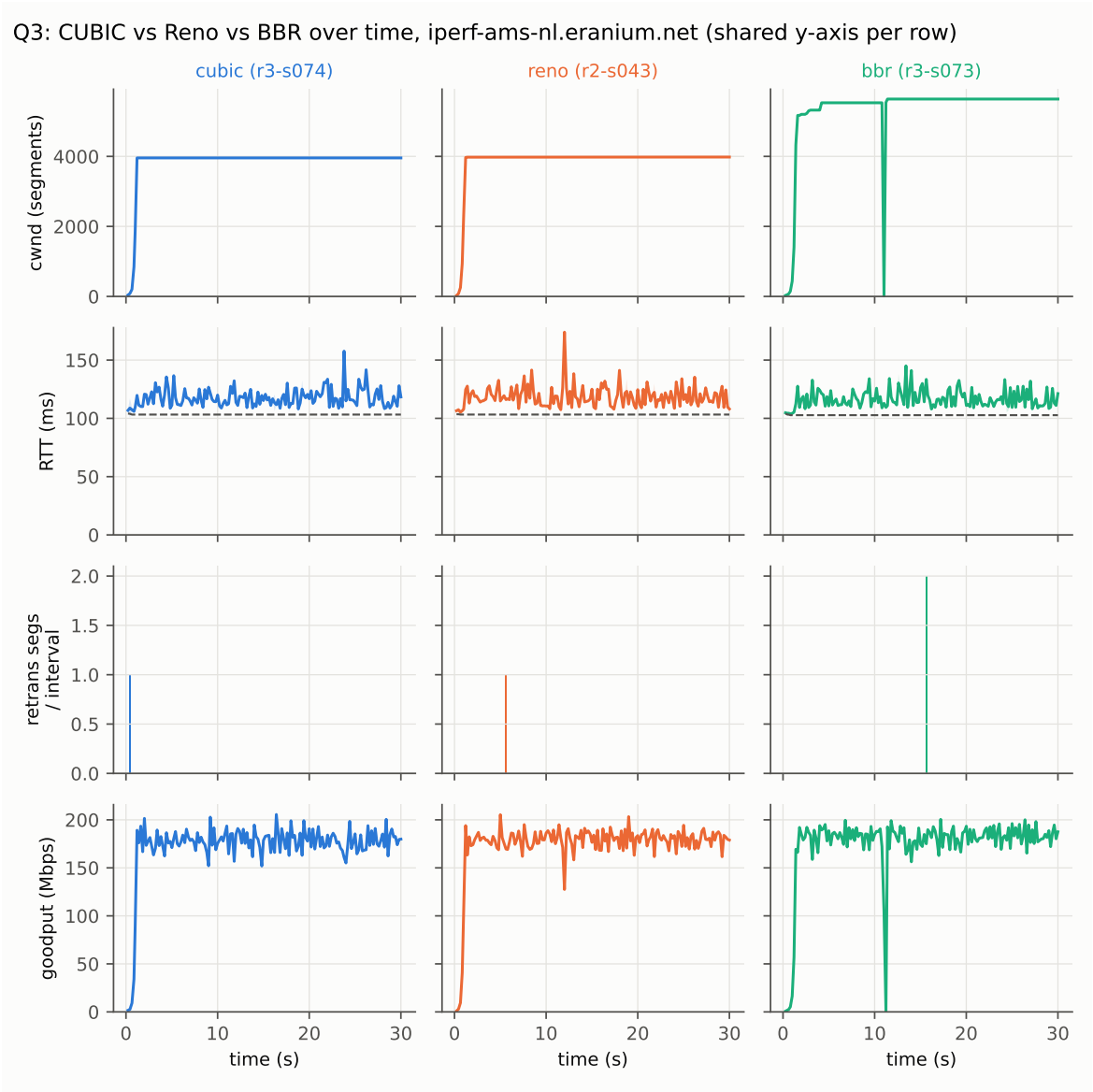


Figure 6: Q3 time series, Amsterdam, median run of each algorithm.

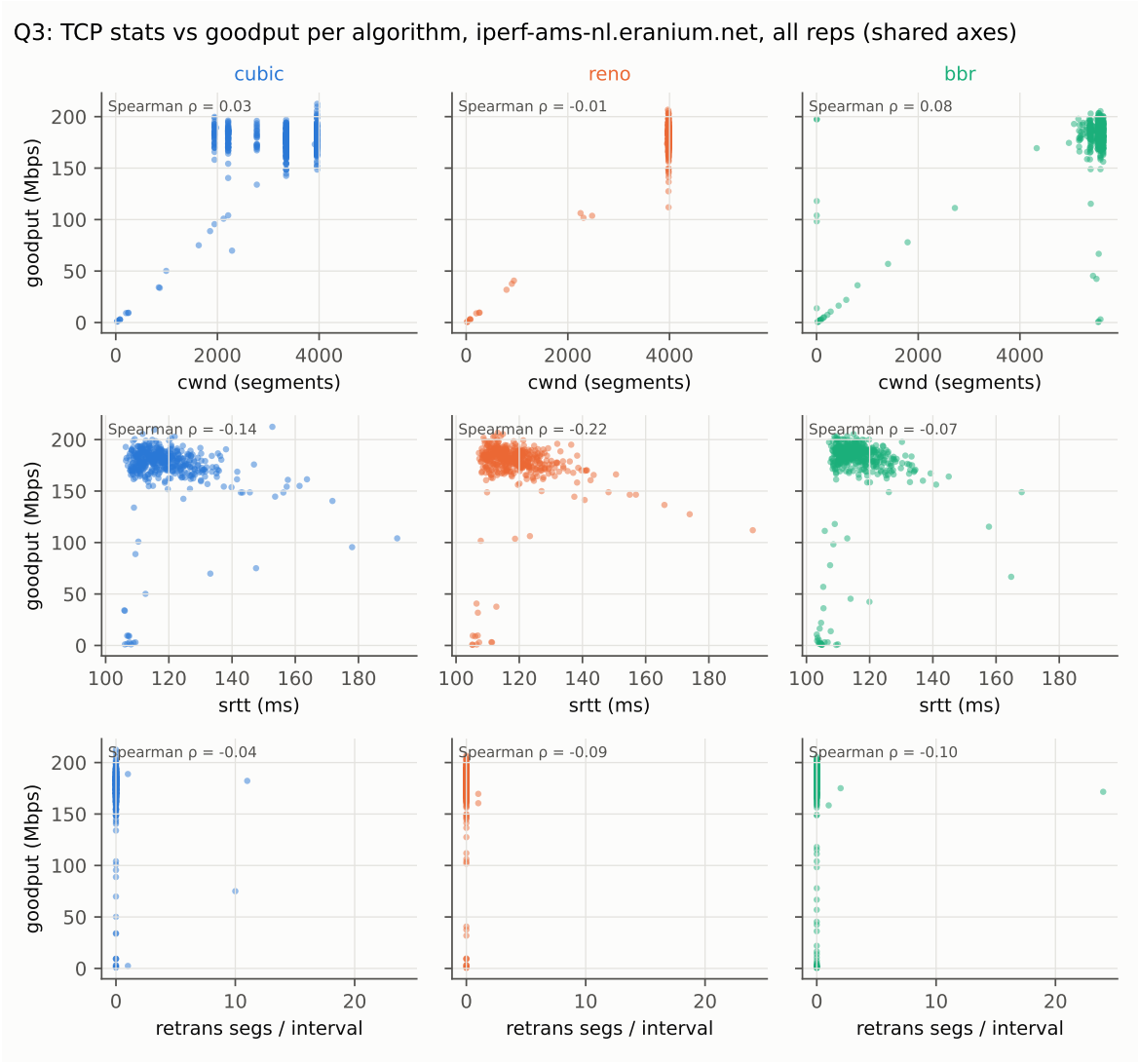


Figure 7: Q3 scatter of cwnd, srtt and retransmits vs goodput, Amsterdam, all reps.

RTT inflation = median srtt / min_rtt; in-flight = median unacked data:

Destination (min RTT)	CC	goodput Mbps (mean ± sd)	RTT inflation	retrans %	in-flight MB
Washington (26 ms)	cubic	234 ± 10	2.90	0.07	2.3
	reno	242 ± 16	3.01	0.06	2.3
	bbr	235 ± 7	2.79	0.02	2.3
Dallas (27 ms)	cubic	154 ± 15	2.09	0.09	1.0
	reno	150 ± 34	1.32	0.20	0.6
	bbr	240 ± 10	2.19	0.17	2.2
Ashburn (27 ms)	cubic	228 ± 13	2.57	0.11	2.2
	reno	235 ± 20	2.52	0.04	2.0
	bbr	240 ± 19	2.11	0.08	1.9
Seattle (53 ms)	cubic	225 ± 4	1.69	0.00	2.5
	reno	225 ± 5	1.70	0.00	2.5
	bbr	224 ± 5	1.58	0.00	2.5
Paris (97 ms)	cubic	185 ± 4	1.12	0.00	2.5

Destination (min RTT)	CC	goodput Mbps (mean ± sd)	RTT inflation	retrans %	in-flight MB
	reno	180 ± 4	1.11	0.00	2.6
	bbr	179 ± 2	1.13	0.00	2.6
Amsterdam (102 ms)	cubic	173 ± 2	1.14	0.00	2.5
	reno	175 ± 2	1.13	0.00	2.6
	bbr	174 ± 1	1.12	0.00	2.7
Naaldwijk (105 ms)	cubic	159 ± 26	1.09	0.00	2.6
	reno	175 ± 2	1.10	0.00	2.6
	bbr	170 ± 5	1.10	0.00	2.7
Zürich (117 ms)	cubic	148 ± 17	1.06	0.00	2.6
	reno	164 ± 2	1.07	0.00	2.7
	bbr	156 ± 4	1.06	0.00	2.7
Sydney (181 ms)	cubic	110 ± 2	1.03	0.02	2.7
	reno	79 ± 29	1.03	0.12	1.3
	bbr	102 ± 2	1.03	0.00	2.7
Singapore (208 ms)	cubic	83 ± 22	1.02	0.00	2.6
	reno	84 ± 9	1.18	0.04	2.7
	bbr	83 ± 5	1.05	0.00	2.7

The host caps in-flight data at ~2.6 MB (Q2c), so a path is link-limited when its BDP is below that cap and sender-limited above it:

1. **Short paths, deep queue** (Washington, Ashburn, Seattle). In-flight exceeds the BDP, so the excess sits in the bottleneck buffer: RTT inflation 1.58–3.01. All three saturate the link (~224–242 Mbps); **BBR has the lowest RTT inflation** on each.
2. **Short lossy path** (Dallas). **BBR 240 Mbps vs CUBIC 154, Reno 150.** On 0.1–0.2% loss, CUBIC and Reno cut cwnd and idle the link (in-flight 0.6–1.0 MB, 14–19% of samples cwnd-limited); BBR does not back off on loss.
3. **Long paths, sender-limited** (Paris → Singapore). The cap is below the BDP, so the link never fills: inflation ~1.0, almost no loss, goodput falls as 1/RTT (180 → 83 Mbps) and the three tie. The algorithm matters only at startup (CUBIC HyStart exited early in 5 of 30 runs) and after rare loss (Sydney Reno 79 ± 29 vs CUBIC 110 ± 2).

Variability. BBR is the most repeatable on long paths (sd 1–5 Mbps); CUBIC's spread comes from HyStart, Reno's from loss bursts; short paths vary 5–20 Mbps with Wi-Fi capacity.

Limitations. One vantage point; single flows (no competition); 3×30 s cannot resolve differences below ~10%; public paths change between runs. "BBR" is Linux BBR v1.

(b) A hand-written algorithm: *QAIMD* (queue-aware AIMD with a BDP anchor)

Motivated by the measurements: keep a small delay-measured queue (vs CUBIC/Reno's 2–3× inflation), back off only on loss with a queue or heavy loss (vs BBR ignoring loss), and jump back toward the BDP after a cut (vs Reno's slow regrowth). **Signals** (all in `TCP_INFO`): `min_rtt` (10 s window), `srtt`, `rtt_round_min`, `btl_bw` (max delivery rate over 10 rounds), per-round loss ratio. BDP = btl_bw × min_rtt; queue q = rtt_round_min − min_rtt.

```
constants: BETA = 0.7, LOSS_TOL = 0.02, MIN_CWND = 4 MSS, PROBE_RTT_EVERY = 10 s
init: state = STARTUP; cwnd = 10 MSS

once per round trip:
    q    = rtt_round_min - min_rtt
    q_thr = clamp(0.15 * min_rtt, 3 ms, 15 ms)
    loss = retrans_bytes_this_round / delivered_bytes_this_round
    congestive_loss = (loss > 0) and (q > q_thr or loss > LOSS_TOL)
```

```
    if state == STARTUP:
        if congestive_loss or (q > 2*q_thr for 2 rounds) or (btl_bw grew < 25% for 3 rounds):
            state = AVOID; cwnd = max(bdp, MIN_CWND)     # drop to pipe size
        else: cwnd = 2 * cwnd
    else:  # AVOID
        if congestive_loss:  cwnd = max(BETA * cwnd, MIN_CWND)
        elif q > q_thr:      cwnd = max(cwnd * (1 - 0.5*(q - q_thr)/srtt), MIN_CWND)
        elif cwnd < bdp:     cwnd = cwnd + max(1 MSS, (bdp - cwnd)/4)
        else:                cwnd = cwnd + 1 MSS

    if min_rtt not refreshed for PROBE_RTT_EVERY:
        hold cwnd = max(0.5*bdp, MIN_CWND) for max(200 ms, 1 srtt)
on RTO: cwnd = MIN_CWND; state = STARTUP capped at the last bdp
```

Design choices. Startup exits to the BDP and regrows by `(bdp−cwnd)/4` (fast recovery, unlike Reno); the delay rule holds the queue near q_{thr} (low RTT inflation); only congestive loss triggers ×0.7 (ignores random loss but still bounds heavy loss, unlike BBR v1).

Fairness and convergence. Below threshold every flow adds the same step per RTT and every reduction is multiplicative, so by the Chiu–Jain argument equal-RTT flows converge to an equal share; the operating point is a small limit cycle at ≈ BDP + a few ms of queue, independent of buffer depth.

Assumptions. A single FIFO bottleneck; `min_rtt` observable every 10 s; app-limited samples flagged; RTT noise below q_{thr}.

Limitations. Short-RTT flows still grow faster (RTT unfairness); noisy delay can cause false back-offs; it yields to loss-based flows that fill the buffer; fairness is argued, not measured.

Reproducibility

- Inputs: `servers.csv`; options in `README.md` (`-n`, `-t`, `-i`, `-r`, `--seed`).
- Dependencies: `Dockerfile` (`FROM ubuntu:24.04`) installs Python 3.12, numpy, pandas, matplotlib.
- Host: Linux with `tcp_bbr` loaded, kernel `6.8.0-147-generic`; `run.sh` runs the image with `--network host --cap-add NET_ADMIN`.
- `./run.sh -n 10 -t 30 -i 0.2 -r 3 --seed 422` runs everything; `./run.sh --analyze-only results/20261021-082724` regenerates plots and tables.

Use of AI tools

We used Claude Code to cross-check the iperf3 protocol against its source, draft the Python modules and plotting code, set up the VM and Docker, and draft this report. We verified it by running against real iperf3 servers, comparing server byte counts with `tcpi_bytes_acked`, and checking the `struct tcp_info` offsets against `linux/tcp.h`.